

Bits, operaciones bit a bit y E/S mapeada en memoria

Sistemas Digitales · Unidad 1: Microcontroladores para sistemas embebidos

Semana 1 · Sesión 2

Universidad Nacional de Loja · Carrera de Computación

Objetivo de la sesión

Aplicar operaciones bit a bit para configurar y controlar pines de E/S mediante los registros DDR, PORT y PIN de un microcontrolador AVR.

Resultado de aprendizaje R1: diseña arquitectura de microcontroladores.

Al terminar podrán:

1. Convertir entre binario, decimal y hexadecimal
2. Encender, apagar o invertir un solo bit
3. Configurar un pin con DDR, PORT y PIN

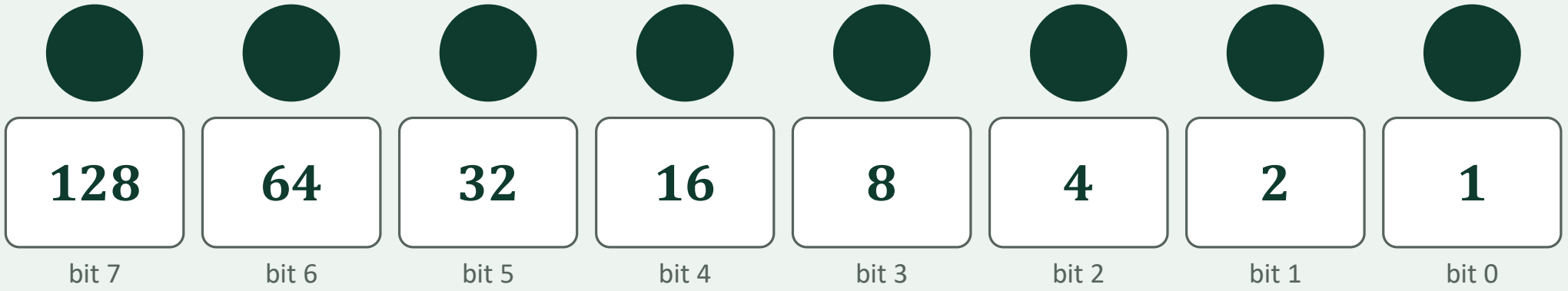
Repaso

PARTICIPACIÓN

1. ¿Qué viaja por el bus de direcciones y qué por el de datos?
2. ¿Dónde guardas un dato que debe sobrevivir a un apagón?
3. ¿Por qué Harvard puede ser más rápida que von Neumann?

El byte humano

PARTICIPACIÓN

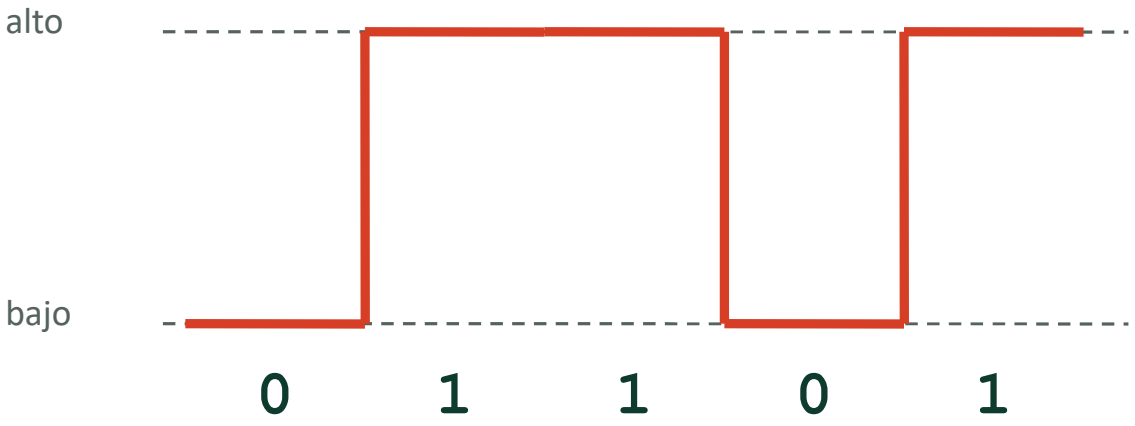


Ocho voluntarios con un cartel cada uno. De pie = 1, sentado = 0.

La clase dicta: 5 · 32 · 0x81 · 255

Reto: ¿quién se para con 0x20 | 0x01?

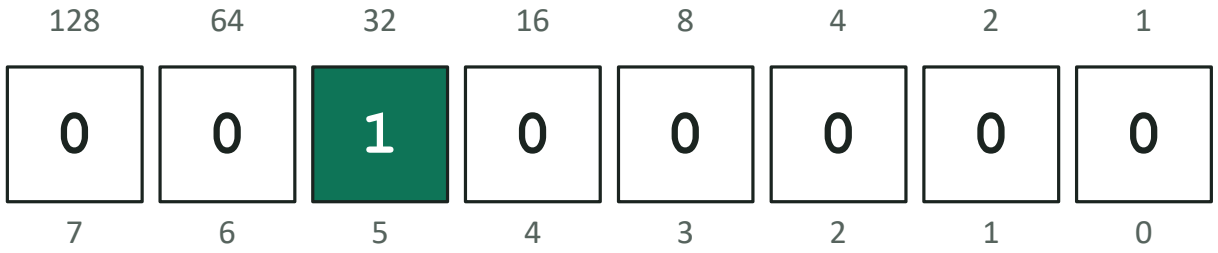
Un bit es un nivel de voltaje



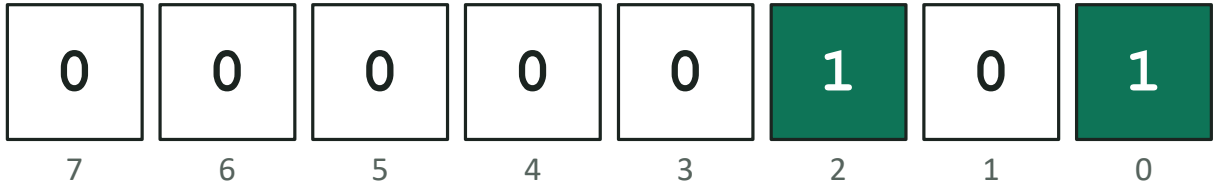
Nivel	Se escribe	Arduino Uno	ESP32
Bajo	0	≈ 0 V	≈ 0 V
Alto	1	≈ 5 V	≈ 3,3 V

Es el transistor como interruptor: en corte da 0, en saturación da 1.

Un byte: ocho bits, cada uno con su peso



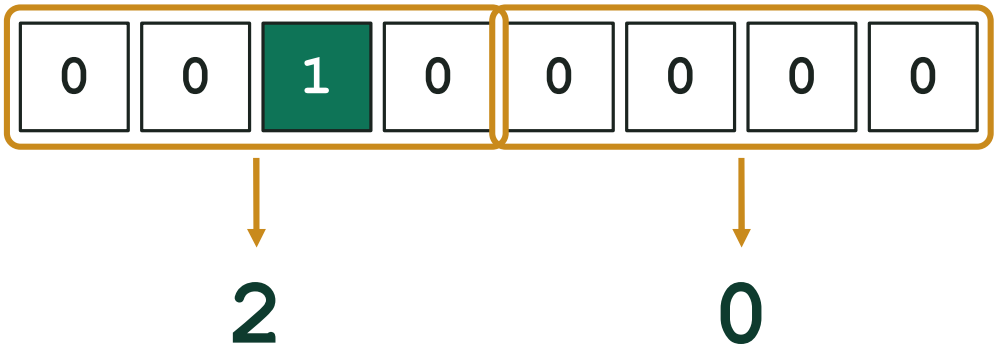
= 32



= 4 + 1 = 5

Los bits se numeran de derecha a izquierda: el bit 0 a la derecha, el bit 7 a la izquierda. El valor es la suma de los pesos de los bits en 1.

Hexadecimal: 4 bits en un solo símbolo



`0010 0000 = 0x20`

El prefijo 0x indica hexadecimal. En C también existe 0b para binario: 0b00100000.

0000 **0**	0001 **1**	0010 **2**	0011 **3**
0100 **4**	0101 **5**	0110 **6**	0111 **7**
1000 **8**	1001 **9**	1010 **A**	1011 **B**
1100 **C**	1101 **D**	1110 **E**	1111 **F**

Convierte en parejas

PARTICIPACIÓN

`1000 0001`

a decimal y a hexadecimal

`0xFF`

a binario y a decimal

`0x08`

¿qué bit está en 1?

`200`

a binario y a hexadecimal

Las compuertas lógicas, un bit a la vez

A	B	AND &	OR |	XOR ^	NOT A ~
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

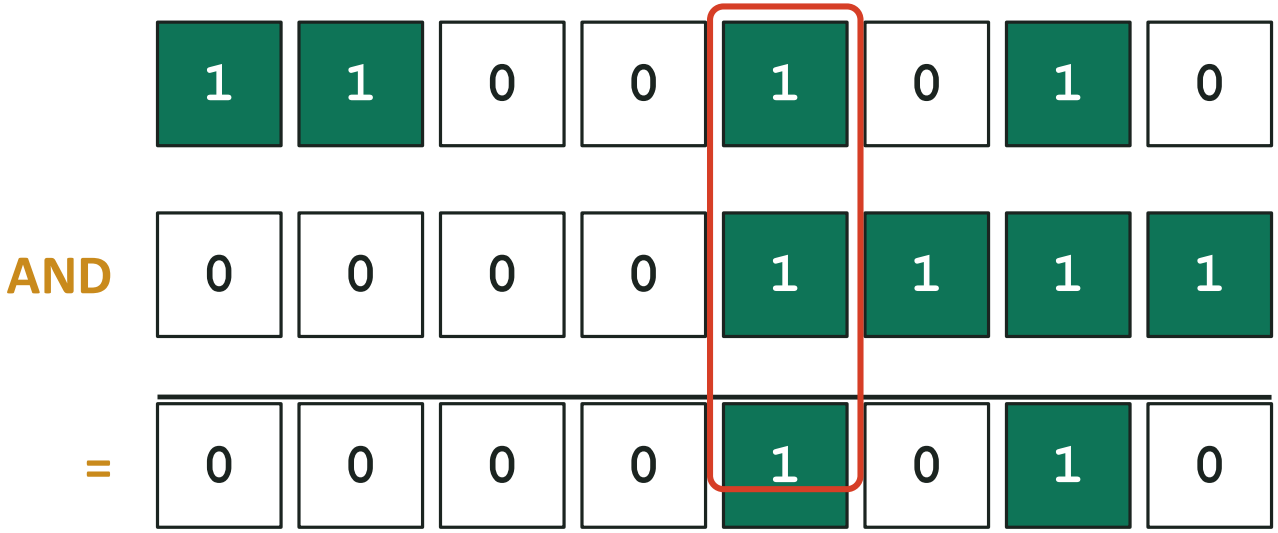
AND: 1 solo si ambos son 1

OR: 1 si alguno es 1

XOR: 1 si son distintos

NOT: invierte

Bit a bit: columna por columna



Cada columna es un AND independiente: aquí, 1 AND 1 = 1.

La CPU hace las 8 columnas a la vez, en una sola instrucción.

```
resultado = 0xCA & 0x0F;    // 0x0A
```

La máscara: 1 << n señala un bit



Una máscara es un byte con 1 solo en los bits que quiero tocar. Escribir 1 << 5 es más legible que 0x20: dice “el bit 5”.

Las cuatro recetas

Quiero	En C	Por qué funciona
Poner el bit n en 1	`REG |= (1<<n);`	OR con 1 da 1; OR con 0 deja igual
Poner el bit n en 0	`REG &= ~(1<<n);`	~ invierte la máscara; AND con 0 da 0
Invertir el bit n	`REG ^= (1<<n);`	XOR con 1 invierte; XOR con 0 deja igual
Preguntar por el bit n	`if (REG & (1<<n))`	Queda distinto de 0 solo si el bit está en 1

En todos los casos, donde la máscara tiene 0 el bit no cambia. REG |= X es la forma corta de REG = REG \| X.

Ejercicios

PARTICIPACIÓN

REG = | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |

Cada ejercicio parte de este valor

1

```
REG |= (1<<0);
```

2

```
REG &= ~(1<<7);
```

3

```
REG ^= 0x0F;
```

4

¿Se cumple if (REG & (1<<3))?

5

Enciende los bits 0 y 1 sin tocar los demás

6

¿Qué falla en REG = (1<<2); para encender el bit 2?

El juego de instrucciones del AVR

Instrucción	Qué hace
`LDI r16, 0x20`	Carga el número 0x20 en el registro r16
`ADD r16, r17`	Suma r17 a r16
`AND  OR  EOR  COM`	AND, OR, XOR y NOT bit a bit: las instrucciones lógicas
`SBI 0x05, 5`	Pone en 1 el bit 5 del registro de E/S 0x05 (PORTB)
`CBI 0x05, 5`	Pone en 0 ese mismo bit
`RJMP`	Salta a otra parte del programa

Demostración: de C a bits

Lenguaje C

```
PORTB |= (1 << 5);
```



Ensamblador

```
sbi   0x05, 5
```



Código máquina en la Flash

```
1001 1010 0010 1101   (0x9A2D)
```

Una línea de C se convierte en una sola instrucción de 2 bytes. Compilado con avr-gcc para el ATmega328P.

RISC y CISC; 8 y 32 bits

RISC

Pocas instrucciones, simples; casi todas en un ciclo de reloj.

AVR, ARM, RISC-V, Xtensa (ESP32)

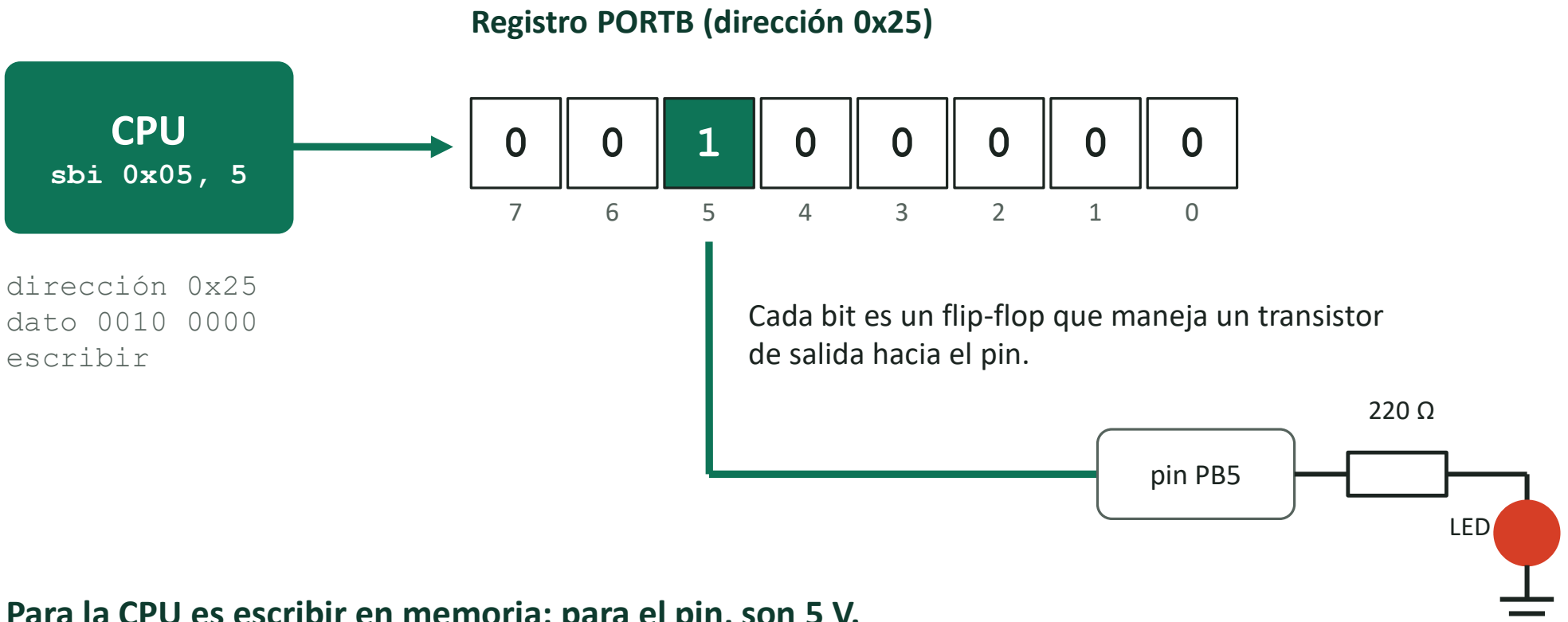
CISC

Muchas instrucciones, algunas muy elaboradas.

x86, el de los PC

8 bits o 32 bits: cuántos bits procesa la CPU de una vez. El ATmega328P suma 8 bits por instrucción; para sumar números de 32 bits necesita 4 sumas. El ESP32 lo hace en una.

La idea central: escribir un bit enciende un LED



Para la CPU es escribir en memoria; para el pin, son 5 V.

Tres registros por puerto

Registro	Para qué sirve	Bit en 1 significa
`DDRB`	Dirección: entrada o salida	Salida
`PORTB`	Salida: el valor que sale. Entrada: activa la pull-up	5 V en el pin, o pull-up activa
`PINB`	Leer el voltaje real del pin	El pin está en alto

El bit n de cada registro corresponde al pin n del puerto. El ATmega328P tiene los puertos B, C y D.

Los cuatro estados de un pin

DDRxn	PORTxn	Estado del pin	Ejemplo
0	0	Entrada flotante: lee al azar	Nada conectado
0	1	Entrada con pull-up	Botón a GND
1	0	Salida en 0 (0 V)	LED apagado
1	1	Salida en 1 (5 V)	LED encendido



Con pull-up, el botón suelto lee 1 y pulsado lee 0: la lógica queda invertida.

Predice el pin

PARTICIPACIÓN

A `DDRB5 = 1` `PORTB5 = 1`

¿Voltaje en el pin?

B `DDRB5 = 1` `PORTB5 = 0`

¿Voltaje en el pin?

C `DDRD2 = 0` `PORTD2 = 1`

Botón pulsado: ¿PIND2?

D `DDRD2 = 0` `PORTD2 = 0`

Nada conectado: ¿PIND2?

Lo mismo con funciones de Arduino

Con funciones de Arduino

```
pinMode(13, OUTPUT);
digitalWrite(13, HIGH);
digitalWrite(13, LOW);
pinMode(2, INPUT_PULLUP);

int b = digitalRead(2);
```

Con registros (lo que pasa por dentro)

```
DDRB  |=  (1 << 5);
PORTB |=  (1 << 5);
PORTB &= ~(1 << 5);
DDRD  &= ~(1 << 2);
PORTD |=  (1 << 2);
int b = (PIND & (1 << 2)) != 0;
```

Las funciones son más legibles y portables; los registros son más rápidos y muestran qué ocurre en el chip.

Resumen

1. Un bit es un nivel de voltaje; un byte son 8 bits; un dígito hex son 4 bits
2. Las operaciones bit a bit aplican AND, OR, XOR y NOT columna por columna
3. Con una máscara 1 << n se toca un solo bit sin cambiar los demás
4. Los periféricos se controlan con registros que tienen dirección, como la memoria
5. DDR elige entrada o salida, PORT pone el valor y PIN lee el pin

Ticket de salida

PARTICIPACIÓN

1. ¿Cuánto es 0x81 en binario?

2. Escribe la línea que apaga el bit 4 de PORTB.

3. Con DDRD2 = 0 y PORTD2 = 1, ¿qué lee PIND2 con el botón pulsado a GND?